Name: Carlos Chipantiza Neptun code: EACN90
Financial Processes (BMETE95MM14), BME, Fall 2026 course id: 54932fw

Homework 4 (Lecture 6): Numerical Greeks

Remark. In this homework we are going to extend the numerical Greeks of Lecture 6: we add the three finite difference methods and the two bump types to the library, and then we measure their errors to choose the bump size. We use only results of the lectures; we state them in boxes with their cell and cite them by number. We need two short new facts (the one-sided second difference and the size of the error), which we deduce from the lecture's own construction and from calculus, with a reference. The code we show is an exact copy of the files, and every number is printed by it (files `*.out`). Files: the library of Lecture 6 (`contract.py`, `enums.py`, and `market_data.py`, `model.py`, `pricer.py` of the `solution` folder) with the NEW CODE at the end of `enums.py` and the NEW VERSION of `Pricer` in `pricer.py`; the runs `q1_fd_greeks_demo.py` and `q1_bump_study.py` (figures in `figures/`); the notebook `4_HW_EXE_1` with the same code. The market data are the csv files of `QuantCourseBP/mkt`. Running example (Lecture 6, cell 22): ATM call on `BLUECHIP_BANK`, $S = K = 130.17$, $T = 1$, `FlatVolModel` ($\sigma = 0.38083$), $r = 0.05$.

1. The assignment.

Statement of the professor (`notebook_flatvol.ipynb`, cell 26).

Introduce two class variables in Pricer that store the finite difference method (forward, backward, central) to be used in numerical greek calculation, and whether the bump size is absolute or relative. Create two Enums to store the labels. Implement all combinations for delta and gamma calculation. Compare greeks using the analytic and the finite difference approximation on a set of European options and visualize the numerical error. Compare absolute and relative bumping and suggest optimal bump sizes. **(15 points)**

2. The problem.

To build	What it must do	Lecture 6 pattern it follows
two Enums	labels FORWARD, BACKWARD, CENTRAL and ABSOLUTE, RELATIVE	`GreekMethod` (`enums.py`)
two class variables	choose the scheme of every numerical Δ and Γ	`relative_bump_size` of `Pricer`
`calc_delta`, `calc_gamma`	3 methods $\times$ 2 bump types (Definitions 1–2, Lemma 1)	central bump of `Pricer` (cell 25)
comparison	error of BUMP against ANALYTIC on a set of European options, plotted	task of cell 25
bump sizes	absolute against relative; the best h	`relative_bump_size` $= 0.01$

The only mathematics we need is the six difference formulas and the size of their error (Section 3); everything else is code in the style of the lecture. Our plan is the following: first we state the results we are going to use, then we write the code and test it (Section 4), and finally we run it on a set of options to measure the errors and to choose the bumps (Section 5). We close with the conclusions (Section 6).

3. Results used.

From the lectures. We start from what the lecture gives us: the difference formulas, the two kinds of bump, the analytic Greeks we compare with, and the model in which we bump.

Definition 1 (Finite differences; Lecture 6, cells 23–24).

Greeks are computed numerically "using finite difference approximation (forward, backward, central)" (cell 23). For a bump $h > 0$:

$$\begin{aligned} f'(x) &\approx \frac{f(x+h) - f(x)}{h} && \text{(forward)}, \\ f'(x) &\approx \frac{f(x) - f(x-h)}{h} && \text{(backward)}, \\ f'(x) &\approx \frac{f(x+h) - f(x-h)}{2h} && \text{(central)}, \end{aligned} \tag{1}$$

and (cell 24) the central second difference, built as a central difference of central differences:

$$f''(x) \approx \frac{f'(x+\frac{h}{2}) - f'(x-\frac{h}{2})}{h} \approx \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}. \tag{2}$$

In the solution, `calc_delta` and `calc_gamma` use the central formulas and `calc_theta` the forward one. For delta and gamma, $f(x) = V(S)$ is the fair value as a function of the spot.

Definition 2 (Absolute and relative bump; Lecture 6, cell 24 and `solution/pricer.py`).

The bump is *absolute* when h is a fixed number (the Δ of cell 24), and *relative* when it is a fixed fraction of the spot,

$$h = \varepsilon\, S, \qquad \varepsilon = \texttt{relative_bump_size} = 0.01 \text{ in the lecture.}$$

Theorem 1 (Black–Scholes delta and gamma; Lecture 3; Lecture 5, `pricer.py`).

With d_1 of the Black–Scholes formula and $\varphi = N'$, for a long European option

$$\Delta_{\text{call}} = N(d_1), \qquad \Delta_{\text{put}} = -N(-d_1), \qquad \Gamma = \frac{\varphi(d_1)}{S\sigma\sqrt{T}}. \tag{3}$$

In the library: `calc_delta`, `calc_gamma` with `GreekMethod.ANALYTIC`.

Definition 3 (Flat volatility model, sticky strike; Lecture 6, cell 18 and `solution/model.py`).

For a contract (K, T) the model reads $\sigma(K, T)$ on the surface and uses it in Black–Scholes. A bumped spot does not change K, so the bumped model reads the same $\sigma(K, T)$ (comment of `FlatVolModel.get_vol`).

Because the FlatVol model keeps $\sigma(K, T)$ fixed when we bump the spot, the bumped prices $V(S \pm h)$ are Black–Scholes prices with the same σ. Therefore Theorem 1 gives exactly the derivative that the differences of Definition 1 approximate, and the difference BUMP − ANALYTIC that we measure is the numerical error only.

New, short results. We need two more facts that the lecture does not state: the one-sided second differences (cell 24 writes only the central one) and the size of the error of each formula. We deduce them here from the lecture's construction and from calculus; references [4], [5].

Lemma 1 (One-sided second differences; as in cell 24).

$$\begin{aligned} f''(x) &\approx \frac{f(x+2h) - 2f(x+h) + f(x)}{h^2} \quad \text{(forward)}, \\ f''(x) &\approx \frac{f(x) - 2f(x-h) + f(x-2h)}{h^2} \quad \text{(backward)}. \end{aligned} \tag{4}$$

Idea. How do we deduce it? From calculus we know that f'' is the derivative of f'. Cell 24 uses exactly this: it obtains the central second difference as a "difference of differences", applying the central formula first to f' and then to f. Since we now want a one-sided version, we can repeat the same construction with the forward formula at both levels; the backward one follows by symmetry.

Proof. We apply the forward difference (1) to f', and then again to f:

$$\begin{aligned} f''(x) &\approx \frac{f'(x+h) - f'(x)}{h} && \text{(forward difference of } f', (1)) \\ &\approx \frac{1}{h}\Big(\frac{f(x+2h) - f(x+h)}{h} - \frac{f(x+h) - f(x)}{h}\Big) && \text{(forward differences of } f) \\ &= \frac{f(x+2h) - 2f(x+h) + f(x)}{h^2}. && \text{(collect the terms)} \end{aligned}$$

Replacing h by $-h$ we obtain the backward formula. □

Lemma 2 (Order of the error; Taylor's formula, [4] Section 4.1).
If f is smooth, the errors of the six formulas are

	forward	backward	central
f'	$+\frac{h}{2}f''(x) + O(h^2)$	$-\frac{h}{2}f''(x) + O(h^2)$	$\frac{h^2}{6}f'''(x) + O(h^4)$
f''	$+h\,f'''(x) + O(h^2)$	$-h\,f'''(x) + O(h^2)$	$\frac{h^2}{12}f''''(x) + O(h^4)$

One-sided schemes are of order 1, central ones of order 2: on a log–log plot of the error against h the slopes are 1 and 2.

Idea. Why do the errors have these sizes? From calculus we know that the derivative is the limit of the difference quotient,

$$f'(x) = \lim_{h\to 0} \frac{f(x+h) - f(x)}{h},$$

so for a small h each formula of Definition 1 is close to $f'(x)$; what we want to know is *how* close, i.e. the first term of the error that does not cancel. Taylor's formula tells us what $f(x+kh)$ is made of: f, then khf', then $\frac{(kh)^2}{2}f''$, and so on, all evaluated at x. Because the forward quotient uses only the points x and $x+h$, after we subtract $f(x)$ and divide by h the first leftover is $\frac{h}{2}f''$: the error is proportional to h. Because the central quotient uses $x+h$ and $x-h$ symmetrically, every even power of h cancels in the subtraction, so the first leftover comes from the h^3 term and, after dividing by $2h$, the error is proportional to h^2. For the second derivative we divide by h^2 instead of h, but the picture is the same: a one-sided formula keeps a term in h, a symmetric one keeps a term in h^2. We now check these claims term by term.

Proof. We insert Taylor's formula $f(x+kh) = f + khf' + \frac{(kh)^2}{2}f'' + \frac{(kh)^3}{6}f''' + \frac{(kh)^4}{24}f'''' + O(h^5)$ (all derivatives at x) into each formula. For the forward f' we take $k = 1$ and $k = 0$:

$$\frac{f(x+h) - f(x)}{h} = f' + \frac{h}{2}f'' + O(h^2).$$

For the central f' we take $k = \pm 1$; since the even terms cancel,

$$\frac{f(x+h) - f(x-h)}{2h} = f' + \frac{h^2}{6}f''' + O(h^4).$$

For the forward f'' we take $k = 2, 1, 0$:

$$f(x+2h) - 2f(x+h) + f(x) = (4-2)\frac{h^2}{2}f'' + (8-2)\frac{h^3}{6}f''' + O(h^4) \quad (f \text{ and } f' \text{ terms cancel})$$
$$= h^2 f'' + h^3 f''' + O(h^4),$$

and we divide by h^2. For the central f'' we take $k = \pm 1, 0$; since now the odd terms cancel,

$$f(x+h) - 2f(x) + f(x-h) = h^2 f'' + \frac{h^4}{12}f'''' + O(h^6),$$

and again we divide by h^2. Replacing h by $-h$ we obtain the backward rows. □

Rounding. Lemma 2 tells us that the smaller h is, the smaller the error. This is true only until the rounding of the computer matters: every fair value is a floating-point number with about 16 correct digits [6]. When h is tiny, the numerator becomes a difference of nearly equal prices and loses digits, and then we divide by h (delta) or by h^2 (gamma), which magnifies the loss. Therefore we expect the error to fall first (Lemma 2) and to rise again afterwards, so there is a best bump size. We are going to read it from the measured error in Section 5; its formula is in the Comment at the end.

4. The code.

We are going to put the two Enums at the end of `enums.py` and to give the class `Pricer` a NEW VERSION in `pricer.py`, with every change marked `# change # k` / `#Delete.` / `# replace by` / `# end change # k`. We keep the lecture's choice as default (CENTRAL, RELATIVE, 1%), so that every result of the lecture stays unchanged.

(a) The labels. We begin with the labels. Since the lecture stores its labels in Enums (`GreekMethod`), we add two more at the end of `enums.py`; they carry no logic.

```
# ================================================================================================
# Homework 4 (NEW CODE): labels of the numerical greeks (finite differences, Lecture 6, cells 23-24).
# ================================================================================================
class FDMethod(str, Enum):
    FORWARD: str = 'FORWARD'
    BACKWARD: str = 'BACKWARD'
    CENTRAL: str = 'CENTRAL'


class BumpType(str, Enum):
    ABSOLUTE: str = 'ABSOLUTE'
    RELATIVE: str = 'RELATIVE'
```

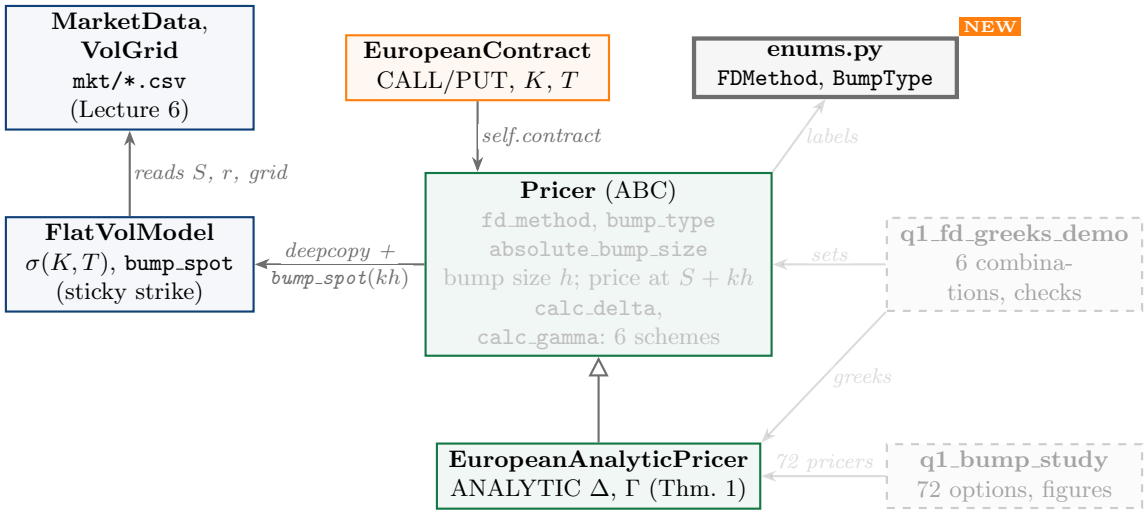


Figure 1: Blocks after (a), the Enums. Bold with NEW: created by this code; UPDATED: changed by this code (orange lines); faded: still to come. Open triangle: inherits from; arrow: uses.

(b) The NEW VERSION of `Pricer` (`pricer.py`). Now we modify `Pricer`. In change #1 we add the two class variables `fd_method` and `bump_type` (Enums of (a)); because an absolute bump needs a

size of its own, we also add `absolute_bump_size`. In change #2 we write two helpers: the bump size h (Definition 2) and the fair value at the bumped spot $S+\text{bump}$, which is the lecture's bump-and-revalue (deep copy of the model, `bump_spot`, `create_pricer`). In change #3 we rewrite `calc_delta` with the three formulas (1), and in change #4 `calc_gamma` with the one-sided formulas (4) of Lemma 1 and the central one (2). Since the analytic pricers inherit both methods (their `BUMP` branch calls `super()`), nothing else has to change.

```
# TASK:
# Implement calc_delta() functions using finite difference approximation
# Homework 4: NEW VERSION of Pricer (changes # 1 - # 4): finite difference method and bump type as class variables,
# all combinations in calc_delta and calc_gamma. The rest of the file is the lecture's solution, unchanged.


class Params:
    pass


class Pricer(ABC):
    # Only for theta calculation via bump and revaluation
    valuation_time: float = 0.0
    relative_bump_size: float = 0.01
    # change # 1 (added: two new class variables and the size of an absolute bump)
    fd_method: FDMethod = FDMethod.CENTRAL          # FORWARD, BACKWARD or CENTRAL difference
    bump_type: BumpType = BumpType.RELATIVE         # h = relative_bump_size * S, or h = absolute_bump_size
    absolute_bump_size: float = 1.0                 # in units of the spot price
    # end change # 1

    def __init__(self, contract: Contract, model: MarketModel, params: Params) -> None:
        self.contract: Contract = contract
        self.model: MarketModel = model
        self.params: Params = params

    @classmethod
    def create_pricer(cls, contract: Contract, model: MarketModel, params: Params) -> Pricer:
        instance = cls(contract, model, params)
        return instance

    @abstractmethod
    def calc_fair_value(self) -> float:
        pass

    # change # 2 (added: the bump size h, and the fair value at the bumped spot S + bump)
    def calc_spot_bump_size(self) -> float:
        if self.bump_type == BumpType.RELATIVE:
            return self.relative_bump_size * self.model.spot
        elif self.bump_type == BumpType.ABSOLUTE:
            return self.absolute_bump_size
        else:
            raise ValueError(f'Unsupported BumpType {self.bump_type}')

    def calc_fair_value_bumped_spot(self, bump: float) -> float:
        model_to_bump = copy.deepcopy(self.model)
        model_to_bump.bump_spot(bump)
        bumped_pricer = self.create_pricer(self.contract, model_to_bump, self.params)
        return bumped_pricer.calc_fair_value()
    # end change # 2

    def calc_delta(self, method: GreekMethod) -> float:
        if method != GreekMethod.BUMP:
            self.raise_unsupported_greek_method_error(method, (GreekMethod.BUMP,))
        # change # 3
        #Delete.bump_size = self.relative_bump_size * self.model.spot
        #Delete.bumped_fair_values = list()
        #Delete.for b in (bump_size, -bump_size):
        #Delete.    model_to_bump = copy.deepcopy(self.model)
        #Delete.    model_to_bump.bump_spot(b)
        #Delete.    bumped_pricer = self.create_pricer(self.contract, model_to_bump, self.params)
        #Delete.    bumped_fair_values.append(bumped_pricer.calc_fair_value())
        #Delete.    del bumped_pricer
        #Delete.    del model_to_bump
        #Delete.return (bumped_fair_values[0] - bumped_fair_values[1]) / (2 * bump_size)
        # replace by
        h = self.calc_spot_bump_size()
        if self.fd_method == FDMethod.FORWARD:
            return (self.calc_fair_value_bumped_spot(h) - self.calc_fair_value()) / h
        elif self.fd_method == FDMethod.BACKWARD:
            return (self.calc_fair_value() - self.calc_fair_value_bumped_spot(-h)) / h
        elif self.fd_method == FDMethod.CENTRAL:
```

```
            return (self.calc_fair_value_bumped_spot(h) - self.calc_fair_value_bumped_spot(-h)) / (2 * h)
        else:
            raise ValueError(f'Unsupported FDMethod {self.fd_method}')
        # end change # 3

    def calc_gamma(self, method: GreekMethod) -> float:
        if method != GreekMethod.BUMP:
            self.raise_unsupported_greek_method_error(method, (GreekMethod.BUMP,))
        # change # 4
        #Delete.bump_size = self.relative_bump_size * self.model.spot
        #Delete.bumped_fair_values = list()
        #Delete.for b in (bump_size, -bump_size):
        #Delete.    model_to_bump = copy.deepcopy(self.model)
        #Delete.    model_to_bump.bump_spot(b)
        #Delete.    bumped_pricer = self.create_pricer(self.contract, model_to_bump, self.params)
        #Delete.    bumped_fair_values.append(bumped_pricer.calc_fair_value())
        #Delete.    del bumped_pricer
        #Delete.    del model_to_bump
        #Delete.return (bumped_fair_values[0] - 2 * self.calc_fair_value() + bumped_fair_values[1]) / (bump_size ** 2)
        # replace by
        h = self.calc_spot_bump_size()
        if self.fd_method == FDMethod.FORWARD:
            return (self.calc_fair_value_bumped_spot(2 * h) - 2 * self.calc_fair_value_bumped_spot(h)
                    + self.calc_fair_value()) / h ** 2
        elif self.fd_method == FDMethod.BACKWARD:
            return (self.calc_fair_value() - 2 * self.calc_fair_value_bumped_spot(-h)
                    + self.calc_fair_value_bumped_spot(-2 * h)) / h ** 2
        elif self.fd_method == FDMethod.CENTRAL:
            return (self.calc_fair_value_bumped_spot(h) - 2 * self.calc_fair_value()
                    + self.calc_fair_value_bumped_spot(-h)) / h ** 2
        else:
            raise ValueError(f'Unsupported FDMethod {self.fd_method}')
        # end change # 4
```

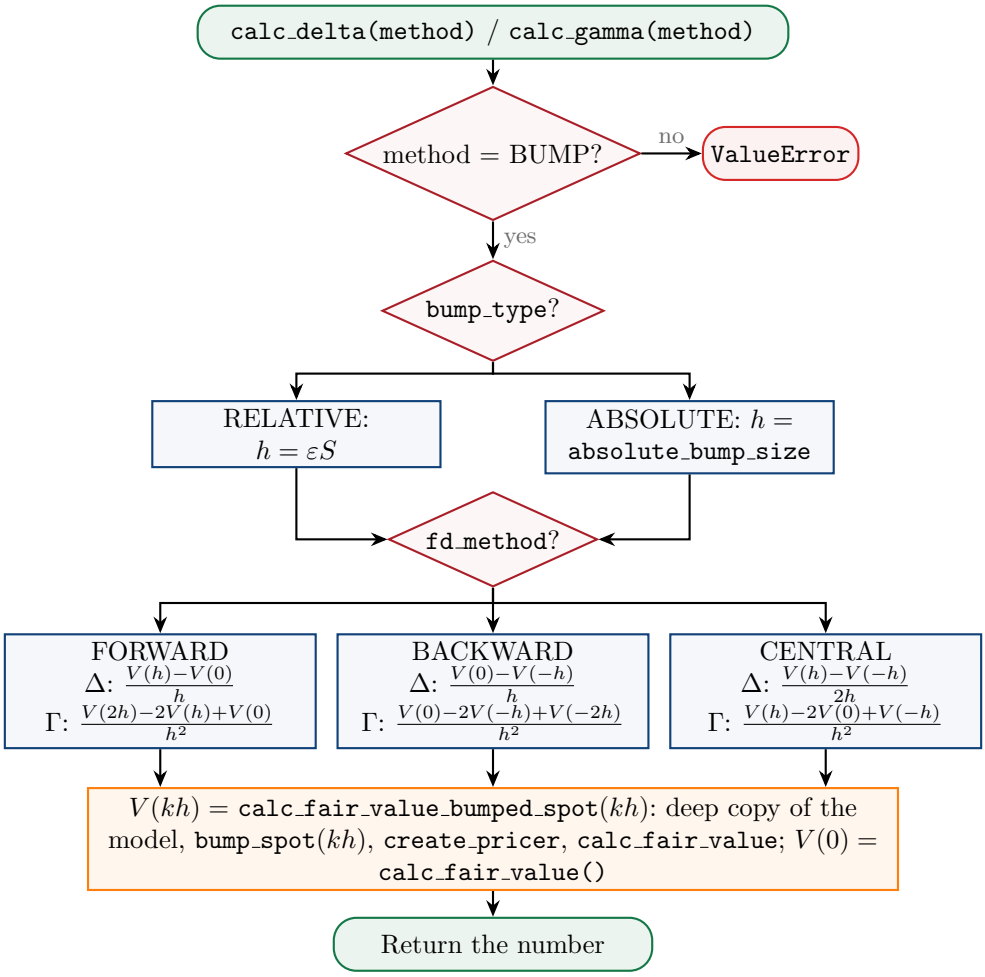


Figure 2: Flowchart of the NEW `calc_delta` and `calc_gamma` (another label raises `ValueError`).

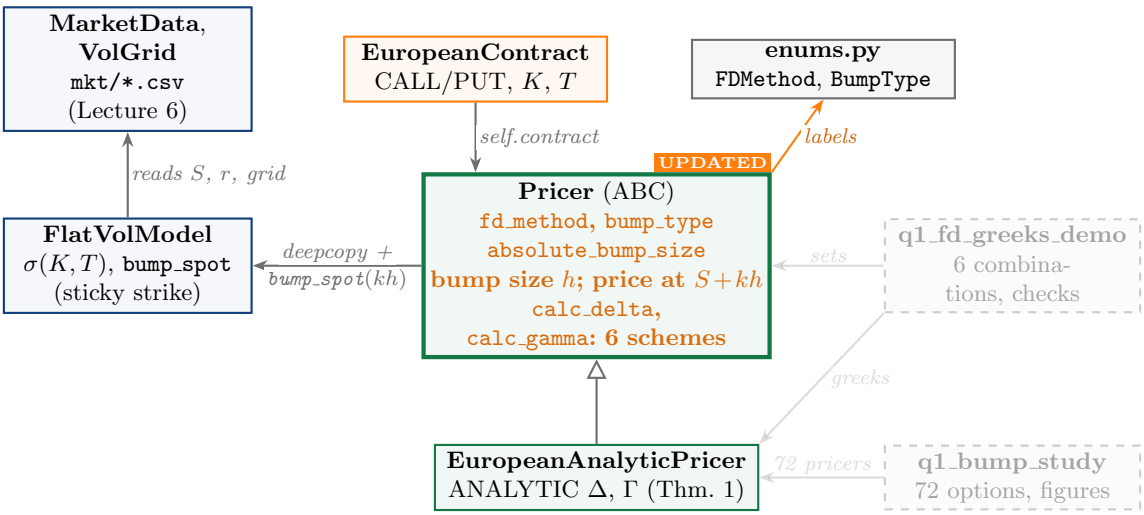


Figure 3: Blocks after (b), the NEW VERSION of `Pricer`. Bold with NEW: created by this code; UPDATED: changed by this code (orange lines); faded: still to come. Open triangle: inherits from; arrow: uses.

(c) Test of the new code. Before we study the errors we test the new code. In the file `q1_fd_greeks_demo.py` we first check that the defaults give the lecture's numbers; then we run the six combinations on the running example, we check two consequences of Lemma 2, we compare a class variable with an instance attribute, and we try a wrong label.

```
# Homework 4: numerical delta and gamma with every finite difference method and bump type (running example).
# Run from this folder with the course venv:  python q1_fd_greeks_demo.py   (printed output: q1_fd_greeks_demo.out)
import sys
from pathlib import Path

here = Path(__file__).resolve().parent
sys.path.insert(0, str(here))                                   # the library of this folder
repo_root = next(p for p in [here, *here.parents] if (p / 'mkt' / 'spot.csv').exists())

from market_data import MarketData
MarketData.market_folder = repo_root / 'mkt'                    # csv files of the repository (Lecture 6)
MarketData.initialize()
from pricer import *                                            # library of Lecture 6 + NEW VERSION of Pricer

# running example: ATM 1Y call on BLUECHIP_BANK, FlatVol model (Lecture 6, cell 22)
und = Stock.BLUECHIP_BANK
spot = MarketData.get_spot()[und]
call = EuropeanContract(und, PutCallFwd.CALL, LongShort.LONG, spot, 1.0)
pricer = EuropeanAnalyticPricer(call, FlatVolModel(und), Params())
delta, gamma = pricer.calc_delta(GreekMethod.ANALYTIC), pricer.calc_gamma(GreekMethod.ANALYTIC)
print(f'ATM call: S = K = {spot}, T = 1, vol = {pricer.model.get_vol(spot, 1.0):.6f}, '
      f'value = {pricer.calc_fair_value():.6f}')
print(f'analytic: delta = {delta:.6f}, gamma = {gamma:.6f}')

# 1. default class variables = the lecture's choice (central, relative 1%): same numbers as the lecture
print(f'\ndefault: {Pricer.fd_method.value}, {Pricer.bump_type.value}, '
      f'relative_bump_size = {Pricer.relative_bump_size}')
d_bump, g_bump = pricer.calc_delta(GreekMethod.BUMP), pricer.calc_gamma(GreekMethod.BUMP)
print(f'bump: delta = {d_bump:.6f}, gamma = {g_bump:.6f}   (lecture: 0.626132, 0.007642)')
assert round(d_bump, 6) == 0.626132 and round(g_bump, 6) == 0.007642

# 2. the six combinations: relative 1% (h = 1.3017) and absolute h = 1
print(f"\n{'method':9s}{'bump':9s}{'h':>8s}{'delta':>11s}{'error':>11s}{'gamma':>11s}{'error':>11s}")
results = {}
for bump_type in BumpType:
    for fd_method in FDMethod:
        Pricer.fd_method, Pricer.bump_type = fd_method, bump_type     # class variables: used by every pricer
        h = pricer.calc_spot_bump_size()
        d, g = pricer.calc_delta(GreekMethod.BUMP), pricer.calc_gamma(GreekMethod.BUMP)
        results[fd_method, bump_type] = (d, g)
        print(f'{fd_method.value:9s}{bump_type.value:9s}{h:8.4f}{d:11.6f}{d - delta:11.2e}{g:11.6f}{g - gamma:11.2e}')
Pricer.fd_method, Pricer.bump_type = FDMethod.CENTRAL, BumpType.RELATIVE  # back to the default

# 3. checks: (forward + backward) / 2 = central for delta; forward and backward delta errors ~ +- h/2 * gamma
for bump_type in BumpType:
    fwd, bwd, cen = (results[m, bump_type][0] for m in FDMethod)
    h = Pricer.relative_bump_size * spot if bump_type == BumpType.RELATIVE else Pricer.absolute_bump_size
    print(f'{bump_type.value:9s}(fwd + bwd)/2 - central = {(fwd + bwd) / 2 - cen: .1e};  '
```

```
          f'fwd error = {fwd - delta:.6f}, h/2 * gamma = {h / 2 * gamma:.6f}')
    assert abs((fwd + bwd) / 2 - cen) < 1e-12

# 4. one pricer only: an instance attribute hides the class variable
pricer.fd_method = FDMethod.FORWARD
other = EuropeanAnalyticPricer(call, FlatVolModel(und), Params())
print(f'\ninstance: {pricer.fd_method.value}, other pricer: {other.fd_method.value}, class: {Pricer.fd_method.value}')
del pricer.fd_method

# 5. a wrong label is refused
Pricer.fd_method = 'SIDEWAYS'
try:
    pricer.calc_delta(GreekMethod.BUMP)
except ValueError as error:
    print('error:', error)
Pricer.fd_method = FDMethod.CENTRAL
```

```
ATM call: S = K = 130.17, T = 1, vol = 0.380830, value = 22.515663
analytic: delta = 0.626163, gamma = 0.007642

default: CENTRAL, RELATIVE, relative_bump_size = 0.01
bump: delta = 0.626132, gamma = 0.007642   (lecture: 0.626132, 0.007642)

method   bump            h      delta      error      gamma      error
FORWARD  ABSOLUTE   1.0000   0.629966   3.80e-03   0.007533  -1.09e-04
BACKWARD ABSOLUTE   1.0000   0.622324  -3.84e-03   0.007750   1.08e-04
CENTRAL  ABSOLUTE   1.0000   0.626145  -1.80e-05   0.007642  -6.19e-08
FORWARD  RELATIVE   1.3017   0.631106   4.94e-03   0.007500  -1.42e-04
BACKWARD RELATIVE   1.3017   0.621159  -5.00e-03   0.007782   1.40e-04
CENTRAL  RELATIVE   1.3017   0.626132  -3.06e-05   0.007642  -1.05e-07
ABSOLUTE (fwd + bwd)/2 - central =  0.0e+00;  fwd error = 0.003803, h/2 * gamma = 0.003821
RELATIVE (fwd + bwd)/2 - central =  0.0e+00;  fwd error = 0.004943, h/2 * gamma = 0.004974

instance: FORWARD, other pricer: CENTRAL, class: CENTRAL
error: Unsupported FDMethod SIDEWAYS
```

Reading the output: the defaults reproduce the lecture (0.626132, 0.007642), so the old results are safe. With the same h, the central error is about 100 times smaller than the one-sided ones, as Lemma 2 announces. Because the forward and backward delta errors are $+\frac{h}{2}\Gamma$ and $-\frac{h}{2}\Gamma$, they have opposite signs and the same size (0.004943 against the predicted 0.004974 for $h = 1.3017$), and their average is exactly the central difference, since the $V(0)$ terms cancel. Finally, setting `Pricer.fd_method` changes every pricer, whereas an attribute set on one pricer changes only that one; this is how we are going to switch schemes in the study.

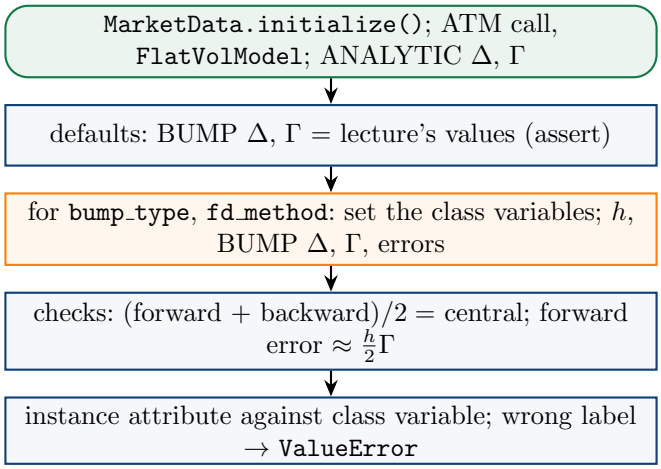


Figure 4: Flowchart of `q1_fd_greeks_demo.py`.

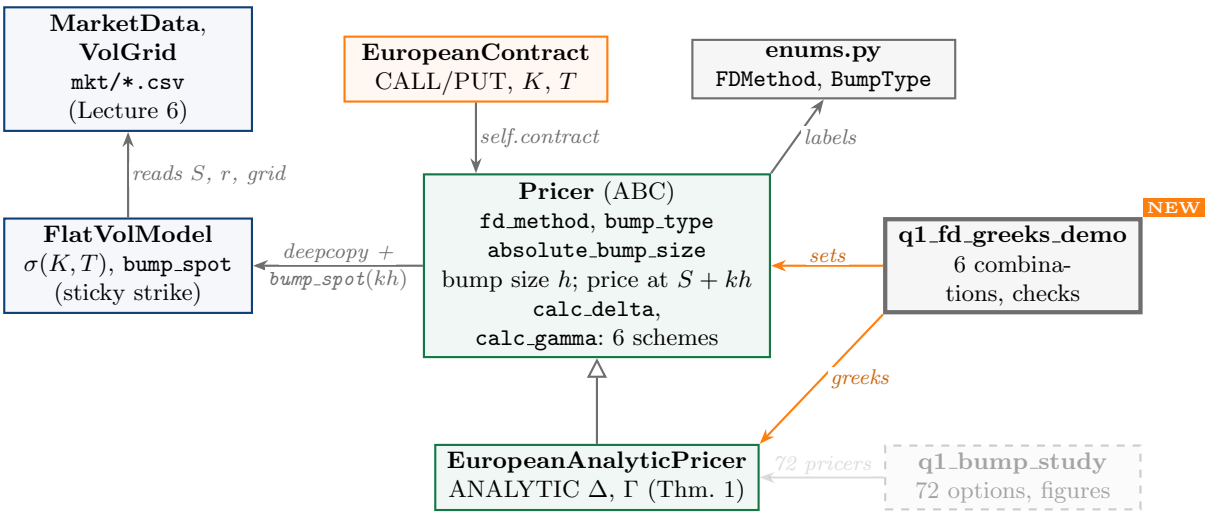


Figure 5: Blocks after (c), `q1_fd_greeks_demo.py`. Bold with NEW: created by this code; UPDATED: changed by this code (orange lines); faded: still to come. Open triangle: inherits from; arrow: uses.

(d) The comparison. Now we can compare. In `q1_bump_study.py` we measure the relative error $|\text{BUMP} - \text{ANALYTIC}|/|\text{ANALYTIC}|$, so that delta and gamma, calls and puts, cheap and expensive stocks are on one scale. In Part 1 we take the running example and 17 relative bumps $\varepsilon = 10^{-1}, 10^{-1.5}, \ldots, 10^{-9}$, for the three methods. In Part 2 we build a set of 72 European options: all four stocks of `mkt/` $\times$ CALL, PUT $\times$ $K/S = 0.9, 1, 1.1 \times T = 0.5, 1, 2$ (`FlatVolModel`); for every method, bump type and bump size we record the worst error over the options of each stock (absolute bumps $h = 10^{1}, \ldots, 10^{-7}$, the same range as εS). In Part 3 we pick the bump with the smallest worst error.

```
# Homework 4: error of the numerical delta and gamma against the bump size, for every finite difference method;
# absolute against relative bumping on a set of European options; suggested (optimal) bump sizes.
# Run from this folder with the course venv:  python q1_bump_study.py   (output: q1_bump_study.out, figures/)
import sys
from pathlib import Path

here = Path(__file__).resolve().parent
sys.path.insert(0, str(here))                                   # the library of this folder
repo_root = next(p for p in [here, *here.parents] if (p / 'mkt' / 'spot.csv').exists())

import matplotlib
matplotlib.use('Agg')                                           # files only (no window)
import matplotlib.pyplot as plt
import numpy as np
from market_data import MarketData
MarketData.market_folder = repo_root / 'mkt'
MarketData.initialize()
from pricer import *                                            # library of Lecture 6 + NEW VERSION of Pricer

(here / 'figures').mkdir(exist_ok=True)
sizes = 10.0 ** -np.arange(1.0, 9.01, 0.5)                      # relative bumps eps = 1e-1, ..., 1e-9 (17 values)
abs_sizes = 10.0 ** -np.arange(-1.0, 7.01, 0.5)                 # absolute bumps h = 1e1, ..., 1e-7 (17 values)
greeks = ('delta', 'gamma')


def set_bump(fd_method: FDMethod, bump_type: BumpType, size: float) -> None:
    Pricer.fd_method, Pricer.bump_type = fd_method, bump_type   # class variables: used by every pricer
    if bump_type == BumpType.RELATIVE:
        Pricer.relative_bump_size = size
    else:
        Pricer.absolute_bump_size = size


def rel_error(pricer: Pricer, greek: str) -> float:
    exact = getattr(pricer, f'calc_{greek}')(GreekMethod.ANALYTIC)
    return abs(getattr(pricer, f'calc_{greek}')(GreekMethod.BUMP) - exact) / abs(exact)


# 1. ATM 1Y call on BLUECHIP_BANK (running example): relative error against eps, 3 methods
und = Stock.BLUECHIP_BANK
spot = MarketData.get_spot()[und]
atm = EuropeanAnalyticPricer(EuropeanContract(und, PutCallFwd.CALL, LongShort.LONG, spot, 1.0),
```

```
                            FlatVolModel(und), Params())
atm_err = {}
for fd_method in FDMethod:
    for greek in greeks:
        errs = []
        for eps in sizes:
            set_bump(fd_method, BumpType.RELATIVE, eps)
            errs.append(rel_error(atm, greek))
        atm_err[fd_method, greek] = np.array(errs)
print('1. ATM call, relative bump: relative error of delta | gamma')
print('eps     ' + ''.join(f'{m.value:>10s}' for m in FDMethod) + ' |' + ''.join(f'{m.value:>10s}' for m in FDMethod))
for i, eps in enumerate(sizes):
    print(f'{eps:7.0e} ' + ''.join(f'{atm_err[m, "delta"][i]:10.1e}' for m in FDMethod) + ' |'
          + ''.join(f'{atm_err[m, "gamma"][i]:10.1e}' for m in FDMethod))
for greek in greeks:                                        # slope of log(error) on eps in [1e-3, 1e-1]
    slopes = [np.polyfit(np.log10(sizes[:5]), np.log10(atm_err[m, greek][:5]), 1)[0] for m in FDMethod]
    print(f'slopes {greek}: ' + ', '.join(f'{m.value} {s:.2f}' for m, s in zip(FDMethod, slopes)))

# 2. the set of European options: 4 stocks x CALL, PUT x K/S = 0.9, 1, 1.1 x T = 0.5, 1, 2 (72 options)
book = [EuropeanAnalyticPricer(EuropeanContract(s, cp, LongShort.LONG, k * MarketData.get_spot()[s], t),
                               FlatVolModel(s), Params())
        for s in Stock for cp in (PutCallFwd.CALL, PutCallFwd.PUT) for k in (0.9, 1.0, 1.1) for t in (0.5, 1.0, 2.0)]
worst = {}                                                  # worst relative error over the options of a stock
for bump_type, grid in ((BumpType.RELATIVE, sizes), (BumpType.ABSOLUTE, abs_sizes)):
    for fd_method in FDMethod:
        for greek in greeks:
            table = np.zeros((len(Stock), len(grid)))
            for j, size in enumerate(grid):
                set_bump(fd_method, bump_type, size)
                for i, stock in enumerate(Stock):
                    table[i, j] = max(rel_error(p, greek) for p in book if p.contract.underlying == stock)
            worst[bump_type, fd_method, greek] = table
set_bump(FDMethod.CENTRAL, BumpType.RELATIVE, 0.01)         # back to the lecture's default
Pricer.absolute_bump_size = 1.0

# 3. suggested bump sizes: the grid value with the smallest worst error over all 72 options
print(f'\n2. {len(book)} options; suggested bumps (worst relative error over the set at that bump)')
print(f"{'greek':6s}{'method':9s}{'relative eps*':>14s}{'error':>9s}{'absolute h*':>13s}{'error':>9s}"
      + ''.join(f'{"h* " + s.value[:8]:>13s}' for s in Stock))
for greek in greeks:
    for fd_method in FDMethod:
        rel = worst[BumpType.RELATIVE, fd_method, greek].max(axis=0)
        ab = worst[BumpType.ABSOLUTE, fd_method, greek]
        per_stock = [abs_sizes[int(np.argmin(row))] for row in ab]
        print(f'{greek:6s}{fd_method.value:9s}{sizes[int(np.argmin(rel))]:14.0e}{rel.min():9.1e}'
              f'{abs_sizes[int(np.argmin(ab.max(axis=0)))]:13.0e}{ab.max(axis=0).min():9.1e}'
              + ''.join(f'{h:13.0e}' for h in per_stock))
print('spots: ' + ', '.join(f'{s.value} {MarketData.get_spot()[s]}' for s in Stock))
for greek in greeks:                                        # the relative eps* does not depend on the stock
    rows = worst[BumpType.RELATIVE, FDMethod.CENTRAL, greek]
    print(f'central {greek}, relative eps* per stock: ' + ', '.join(f'{sizes[int(np.argmin(r))]:.0e}' for r in rows))

# one relative bump for both greeks (the library has one relative_bump_size): central differences
both = np.maximum(worst[BumpType.RELATIVE, FDMethod.CENTRAL, 'delta'].max(axis=0),
                  worst[BumpType.RELATIVE, FDMethod.CENTRAL, 'gamma'].max(axis=0))
best = int(np.argmin(both))
lecture = int(np.argmin(abs(sizes - 0.01)))
print(f'central, one eps for delta and gamma: eps* = {sizes[best]:.0e}, worst error {both[best]:.1e}  '
      f'(lecture eps = 1e-02: {both[lecture]:.1e})')

# 4. data for the figures of the report (csv) and the figures (png)
np.savetxt(here / 'figures' / 'atm_errors.csv',
           np.column_stack([sizes] + [atm_err[m, g] for g in greeks for m in FDMethod]), delimiter=',',
           header='eps,' + ','.join(f'{g}_{m.value}' for g in greeks for m in FDMethod), comments='')
for bump_type, grid in ((BumpType.RELATIVE, sizes), (BumpType.ABSOLUTE, abs_sizes)):
    for greek in greeks:
        np.savetxt(here / 'figures' / f'worst_{greek}_{bump_type.value.lower()}_central.csv',
                   np.column_stack([grid, worst[bump_type, FDMethod.CENTRAL, greek].T]), delimiter=',',
                   header='size,' + ','.join(s.value for s in Stock), comments='')

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
for ax, greek in zip(axes, greeks):
    for m, mark in zip(FDMethod, 'o^s'):
        ax.loglog(sizes, atm_err[m, greek], marker=mark, label=m.value.lower())
    ax.set_title(f'{greek}: ATM 1Y call, BLUECHIP_BANK, relative bump')
    ax.set_xlabel('relative bump eps (h = eps * S)')
    ax.set_ylabel('relative error')
    ax.grid(True, which='major', ls=':')
    ax.legend()
fig.tight_layout()
fig.savefig(here / 'figures' / 'q1_error_vs_bump.png', dpi=150)
```

```
fig, axes = plt.subplots(2, 2, figsize=(11, 7.5))
for row, greek in enumerate(greeks):
    for col, (bump_type, grid) in enumerate(((BumpType.ABSOLUTE, abs_sizes), (BumpType.RELATIVE, sizes))):
        ax = axes[row, col]
        for i, s in enumerate(Stock):
            ax.loglog(grid, worst[bump_type, FDMethod.CENTRAL, greek][i], marker='o', ms=3,
                      label=f'{s.value} (S = {MarketData.get_spot()[s]})')
        ax.set_title(f'central {greek}, {bump_type.value.lower()} bump: worst error per stock')
        ax.set_xlabel('h' if bump_type == BumpType.ABSOLUTE else 'eps (h = eps * S)')
        ax.set_ylabel('worst relative error')
        ax.grid(True, which='major', ls=':')
        ax.legend(fontsize=7)
fig.tight_layout()
fig.savefig(here / 'figures' / 'q1_absolute_vs_relative.png', dpi=150)
print('figures saved in figures/')
```

```
1. ATM call, relative bump: relative error of delta | gamma
eps        FORWARD  BACKWARD   CENTRAL |   FORWARD  BACKWARD   CENTRAL
  1e-01    7.4e-02   8.4e-02   4.8e-03 |   1.9e-01   1.6e-01   1.4e-03
  3e-02    2.5e-02   2.6e-02   4.9e-04 |   5.9e-02   5.7e-02   1.4e-04
  1e-02    7.9e-03   8.0e-03   4.9e-05 |   1.9e-02   1.8e-02   1.4e-05
  3e-03    2.5e-03   2.5e-03   4.9e-06 |   5.8e-03   5.8e-03   1.4e-06
  1e-03    7.9e-04   7.9e-04   4.9e-07 |   1.8e-03   1.8e-03   1.4e-07
  3e-04    2.5e-04   2.5e-04   4.9e-08 |   5.8e-04   5.8e-04   1.3e-08
  1e-04    7.9e-05   7.9e-05   4.9e-09 |   1.8e-04   1.8e-04   5.0e-09
  3e-05    2.5e-05   2.5e-05   4.9e-10 |   5.8e-05   5.8e-05   1.3e-07
  1e-05    7.9e-06   7.9e-06   5.6e-11 |   1.9e-05   1.7e-05   1.1e-06
  3e-06    2.5e-06   2.5e-06   2.7e-11 |   1.4e-05   1.9e-05   8.2e-06
  1e-06    7.9e-07   7.9e-07   8.8e-13 |   4.1e-05   6.9e-05   4.1e-05
  3e-07    2.5e-07   2.5e-07   1.7e-10 |   1.8e-04   1.8e-04   3.7e-04
  1e-07    7.9e-08   8.2e-08   1.7e-09 |   2.1e-02   2.3e-02   9.7e-03
  3e-08    2.8e-08   2.5e-08   1.7e-09 |   1.2e-01   9.7e-02   4.3e-02
  1e-08    2.0e-08   6.0e-09   7.1e-09 |   1.5e+00   6.5e-01   6.5e-01
  3e-09    4.7e-08   8.0e-09   2.0e-08 |   1.7e+01   4.5e+00   1.0e+01
  1e-09    2.4e-07   6.4e-08   1.5e-07 |   3.9e+02   2.2e+02   1.1e+02
slopes delta: FORWARD 0.99, BACKWARD 1.01, CENTRAL 2.00
slopes gamma: FORWARD 1.00, BACKWARD 0.98, CENTRAL 2.01

2. 72 options; suggested bumps (worst relative error over the set at that bump)
greek method    relative eps*    error  absolute h*    error  h* TEST_COM  h* BLUECHIP  h* TIPTOP_S  h* MONEY_MA
delta FORWARD           1e-08  6.7e-08        3e-06  7.9e-08        1e-06        1e-06        1e-06        3e-06
delta BACKWARD          1e-08  5.3e-08        3e-06  8.2e-08        1e-06        1e-06        1e-06        3e-06
delta CENTRAL           3e-06  8.0e-11        3e-04  1.3e-10        3e-04        3e-04        1e-03        1e-03
gamma FORWARD           1e-05  3.8e-05        1e-03  3.8e-05        3e-04        1e-03        1e-03        3e-03
gamma BACKWARD          1e-05  4.1e-05        1e-03  4.1e-05        3e-04        1e-03        1e-03        1e-03
gamma CENTRAL           1e-04  4.1e-08        3e-02  1.4e-07        1e-02        1e-02        1e-02        3e-02
spots: TEST_COMPANY 100.0, BLUECHIP_BANK 130.17, TIPTOP_SOLUTIONS 177.32, MONEY_MALL 250.86
central delta, relative eps* per stock: 3e-06, 3e-06, 3e-06, 3e-06
central gamma, relative eps* per stock: 1e-04, 1e-04, 1e-04, 1e-04
central, one eps for delta and gamma: eps* = 1e-04, worst error 4.1e-08  (lecture eps = 1e-02: 3.2e-04)
figures saved in figures/
```

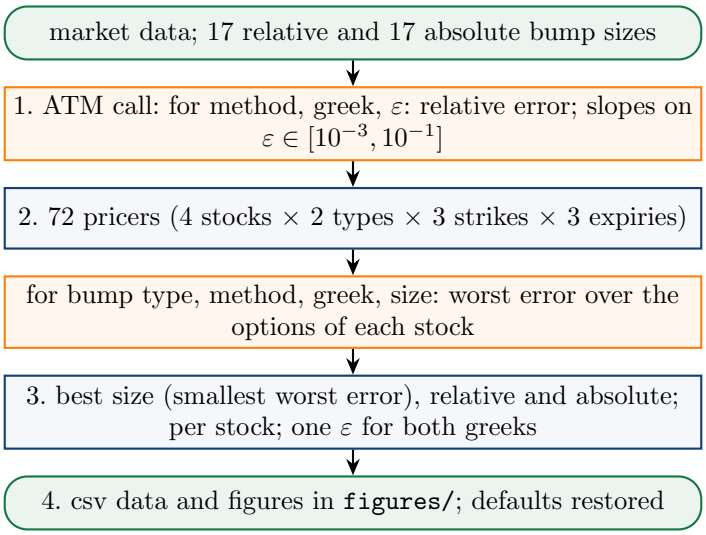


Figure 6: Flowchart of `q1_bump_study.py`.

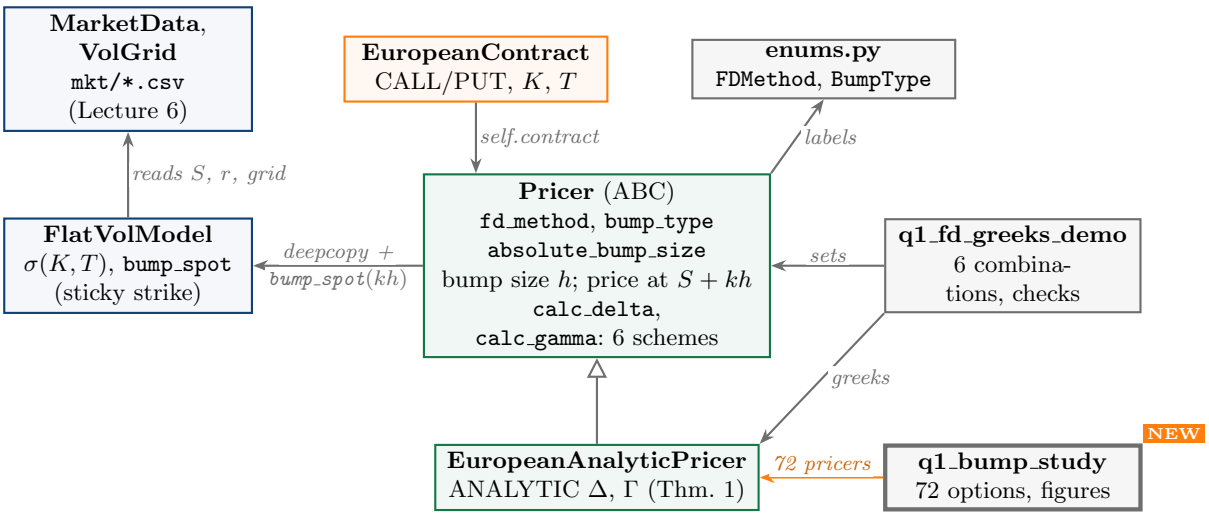


Figure 7: Blocks after (d), `q1_bump_study.py`: all blocks of the homework. Bold with NEW: created by this code; UPDATED: changed by this code (orange lines); faded: still to come. Open triangle: inherits from; arrow: uses.

5. Results.

The numerical error. We look first at the running example (Figure 8).

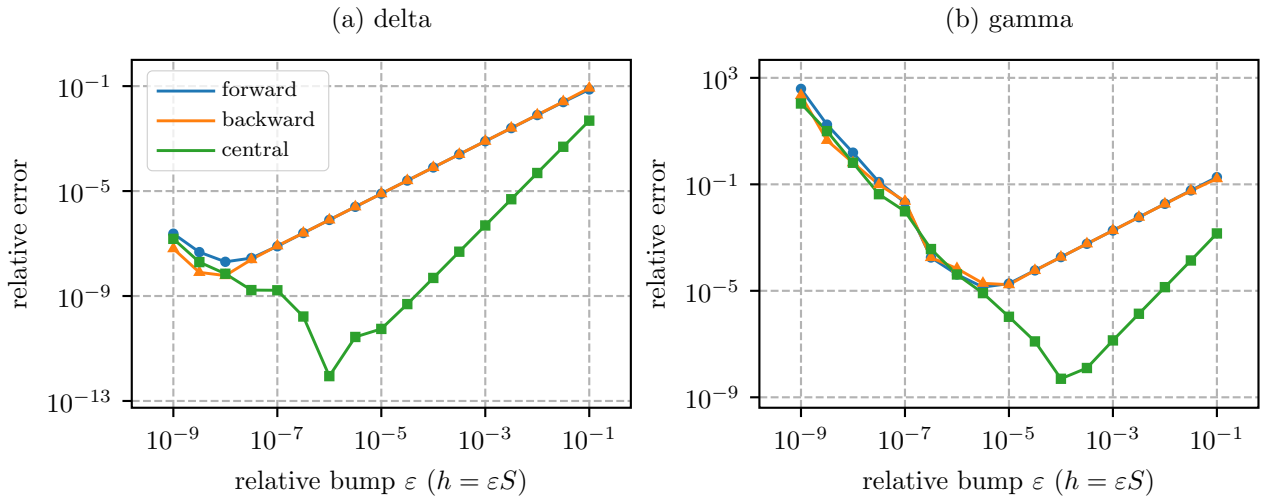


Figure 8: Relative error of the numerical delta and gamma of the running example (ATM 1Y call, `BLUECHIP_BANK`, relative bump). Right part: truncation, slope 1 (forward, backward) and 2 (central); left part: rounding. Same numbers as `figures/q1_error_vs_bump.png`.

On the right of the figure the slopes are 0.99, 1.01, 2.00 (delta) and 1.00, 0.98, 2.01 (gamma), exactly as Lemma 2 predicts. Moving to the left, the error falls until the rounding takes over (Section 3) and then rises; for gamma, which divides by h^2, the rise is much steeper. At every reasonable ε the central scheme is the most accurate, by orders of magnitude. The lecture's $\varepsilon = 1\%$ gives $4.9 \cdot 10^{-5}$ (delta) and $1.4 \cdot 10^{-5}$ (gamma), whereas the best central bumps give about 10^{-12} and $5 \cdot 10^{-9}$.

Absolute against relative bumping. Next we compare the two bump types on the set of 72 options, with the central scheme (Figure 9).

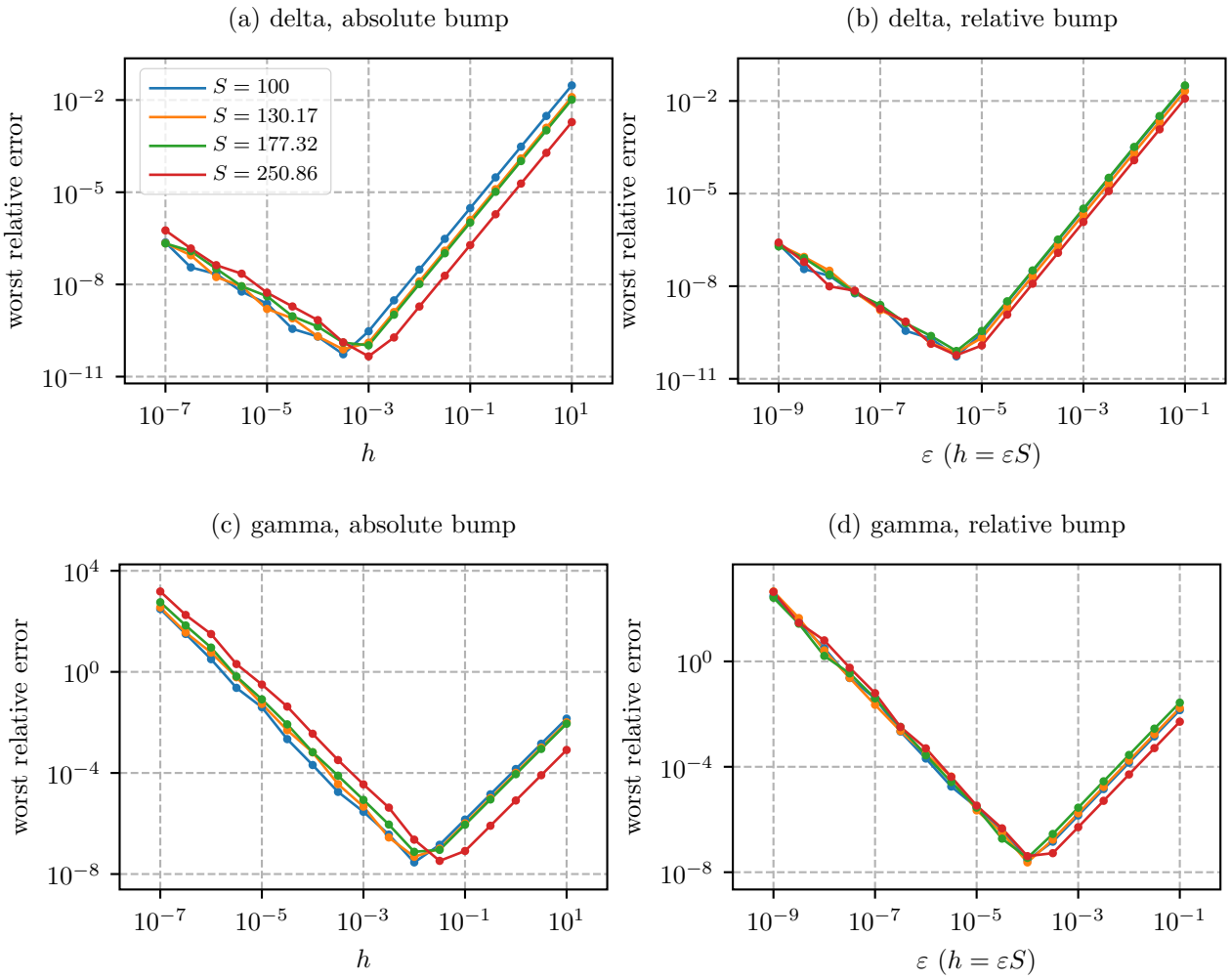


Figure 9: Worst relative error of the central delta and gamma over the 18 options of each stock (`TEST_COMPANY` $S = 100$, `BLUECHIP_BANK` 130.17, `TIPTOP_SOLUTIONS` 177.32, `MONEY_MALL` 250.86). Left: absolute bump h; right: relative bump ε. Same numbers as `figures/q1_absolute_vs_relative.png`.

With an absolute bump the curves are shifted along the axis according to the spot, so the best h depends on the stock: $3 \cdot 10^{-4}$ ($S = 100, 130$) against 10^{-3} ($S = 177, 251$) for delta, and 10^{-2} ($S = 100, 130, 177$) against $3 \cdot 10^{-2}$ ($S = 251$) for gamma. With a relative bump, however, the four curves have their minimum at the same place, $\varepsilon^* = 3 \cdot 10^{-6}$ (delta) and 10^{-4} (gamma), for every stock (output, Part 3). This is what we expected: the same $h = 1$ is 1% of $S = 100$ but only 0.4% of $S = 250.86$, while the same ε is the same fraction of every spot. Therefore one relative size serves the whole book, whereas an absolute size has to be tuned for each underlying. We also see that the best single absolute bump is less accurate on the set: $1.3 \cdot 10^{-10}$ against $8.0 \cdot 10^{-11}$ (delta), $1.4 \cdot 10^{-7}$ against $4.1 \cdot 10^{-8}$ (gamma).

Suggested bump sizes. Finally we collect the best bumps (Part 3 of the output; worst relative error over the 72 options).

Greek	method	relative ε^*	worst error	absolute h^* (best for all)	worst error
delta	forward	10^{-8}	$6.7 \cdot 10^{-8}$	$3 \cdot 10^{-6}$	$7.9 \cdot 10^{-8}$
delta	backward	10^{-8}	$5.3 \cdot 10^{-8}$	$3 \cdot 10^{-6}$	$8.2 \cdot 10^{-8}$
delta	central	$3 \cdot 10^{-6}$	$8.0 \cdot 10^{-11}$	$3 \cdot 10^{-4}$	$1.3 \cdot 10^{-10}$
gamma	forward	10^{-5}	$3.8 \cdot 10^{-5}$	10^{-3}	$3.8 \cdot 10^{-5}$
gamma	backward	10^{-5}	$4.1 \cdot 10^{-5}$	10^{-3}	$4.1 \cdot 10^{-5}$
gamma	central	10^{-4}	$4.1 \cdot 10^{-8}$	$3 \cdot 10^{-2}$	$1.4 \cdot 10^{-7}$

Answer. We recommend central differences with a relative bump. The best sizes are $\varepsilon^* = 3 \cdot 10^{-6}$ for delta and $\varepsilon^* = 10^{-4}$ for gamma, the same for every stock. Since the library has a single `relative_bump_size`, we suggest the common value $\varepsilon = 10^{-4}$, with worst relative error $4.1 \cdot 10^{-8}$ on the 72 options, against $3.2 \cdot 10^{-4}$ for the lecture's 1%. Forward and backward differences are first order (Lemma 2) and much less accurate; if they must be used, we would take $\varepsilon \approx 10^{-8}$ (delta) and $\varepsilon \approx 10^{-5}$ (gamma). An absolute bump works only if it is tuned for each spot level (Figure 9).

6. Conclusions.

- *The library.* With two Enums, two class variables and two helpers we obtained all six schemes for delta and gamma inside `Pricer`; the analytic pricers inherit them without any change, and with the default labels the library gives exactly the numbers of the lecture (Figure 7).
- *The errors behave as calculus predicts.* On the log–log plots the one-sided schemes have slope 1 and the central ones slope 2 (Lemma 2), until the rounding of the computer takes over and the error grows again. Since the central delta needs two prices like the forward one, and the central gamma three prices like the one-sided ones, the central scheme is more accurate by orders of magnitude at the same cost; there is no reason to use the one-sided ones for delta and gamma.
- *Relative bumping is the right choice for a library.* Because the error depends on h/S and not on h, the same relative ε is optimal for every stock, whereas an absolute h has to be tuned for each spot level and is less accurate on the whole set.
- *Suggested bumps.* For the central scheme, $\varepsilon^* = 3 \cdot 10^{-6}$ (delta) and $\varepsilon^* = 10^{-4}$ (gamma); with one common value, $\varepsilon = 10^{-4}$, the worst relative error on the 72 options is $4.1 \cdot 10^{-8}$, four orders of magnitude below the lecture's 1% ($3.2 \cdot 10^{-4}$). Gamma needs a larger bump than delta because it divides by h^2, so the rounding error hurts it earlier; this is the rule $\varepsilon^* \sim u^{1/3}$ for delta and $u^{1/4}$ for gamma of the Comment.
- *What stays open.* We measured the error only where we know the exact answer (European options in the FlatVol model). For a pricer without a formula (trees, Monte Carlo, PDE) the same study can be run against a fine reference, and the best bump may be larger, since the prices themselves carry a numerical error bigger than the rounding.

Comment (to keep in mind: where the best bump comes from).

- *A formula for the optimum* ([4] Section 4.1, [5] Section 8.1). If every price carries a rounding error of at most $u|V|$, with $u = 2^{-53} \approx 1.1 \cdot 10^{-16}$ [6], the total error of the central delta is at most $E(h) = \frac{h^2}{6}|V'''| + \frac{u|V|}{h}$ (Lemma 2 plus the two rounding errors divided by $2h$), smallest at $h^* = (3u|V|/|V'''|)^{1/3}$. The same balance gives $h^* = 2(u|V|/|V''|)^{1/2}$ for the forward delta, $(8u|V|/|V'''|)^{1/3}$ for the forward gamma and $(48u|V|/|V''''|)^{1/4}$ for the central gamma. For the running example ($V = 22.5157$, $V'' = \Gamma = 0.007642$, $|V'''| = 1.083 \cdot 10^{-4}$, $|V''''| = 7.43 \cdot 10^{-7}$) the predicted $\varepsilon^* = h^*/S$ are $8.8 \cdot 10^{-9}$, $3.2 \cdot 10^{-6}$, $4.4 \cdot 10^{-6}$ and $1.5 \cdot 10^{-4}$: the same orders as the measured minima 10^{-8}, 10^{-6}, $3 \cdot 10^{-6}$, 10^{-4} (Figure 8). This is why $\varepsilon^* \sim u^{1/2}$, $u^{1/3}$, $u^{1/4}$: the more the formula divides by h, the larger the best bump.
- *Lecture code.* The numerical vega of the solution is 0 (the deep copy separates the grid values from their interpolator); it is not part of this assignment and is left as in the lecture.

References.

[1] Lecture 6 of this course: `notebook_flatvol.ipynb` (cells 18, 22–26) and the `solution` folder (`market_data.py`, `model.py`, `pricer.py`).

[2] Lecture 5 of this course: `notebook_building_blocks.ipynb` (the library: contract, model, pricer; `GreekMethod`).

[3] Lecture 3 of this course: `presentation_03_option_basics_and_bsm_model_2026.ipynb` (Black–Scholes formula and Greeks).

[4] R. L. Burden, J. D. Faires, *Numerical Analysis*, 9th ed., Brooks/Cole, 2011, Section 4.1 (numerical differentiation: forward, backward and centred formulas, their errors, and the round-off error).

[5] J. Nocedal, S. J. Wright, *Numerical Optimization*, 2nd ed., Springer, 2006, Section 8.1 (finite-difference derivative approximations and the choice of h).

[6] D. Goldberg, What every computer scientist should know about floating-point arithmetic, *ACM Computing Surveys* 23(1), 1991, 5–48 (double precision, unit round-off 2^{-53}).